Tarea 1: Clasificación lineal con Keras

Regresión logística ponderada sobre el dataset Wine (UCI) · Diplomado MLDS-5, Módulo 5 Unidad 1

Natalia Remolina Rodríguez · 15 de octubre de 2026

1. Resumen

Se entrenó una **regresión logística** en Keras para distinguir dos tipos de vino (clases 2 y 3 del dataset Wine), compensando el desbalance entre clases con una **entropía cruzada ponderada**. Después de 10 épocas con el optimizador Adam (tasa de aprendizaje 0.5) el modelo obtuvo:

Indicador	Resultado
Pruebas del grader UNCode (Test 1_1 a 6_2)	12 de 12 con nota 100
Accuracy en entrenamiento (83 muestras)	98.8 % (82 de 83)
Accuracy en prueba (36 muestras)	91.7 % (33 de 36)
Pérdida final (época 10)	0.0380 (inició en 0.9400)

2. El problema

El dataset Wine tiene 178 vinos descritos por 13 variables químicas (alcohol, ácido málico, flavonoides, intensidad de color, prolina, etc.). Para la tarea solo se usan dos clases, recodificadas como 0 y 1, lo que deja 119 muestras. Las variables se escalan a [0, 1] con MinMaxScaler y se separa 70 % para entrenamiento y 30 % para prueba de forma estratificada.

Partición	Clase 0 (vino tipo 2)	Clase 1 (vino tipo 3)	Total
Entrenamiento	50	33	83
Prueba	21	15	36

La clase 1 tiene un 34 % menos de muestras que la clase 0. Si no se corrige, el modelo tiende a favorecer a la clase mayoritaria. La solución que pide la tarea es darle más peso en la función de pérdida a los errores de la clase minoritaria.

3. Cómo se abordó

La tarea es incremental: cada función es una pieza del pipeline y la siguiente la usa. La estrategia fue:

1. Resolver las funciones en orden y comparar cada una contra la "Salida esperada" del enunciado antes de avanzar.

2. Leer las celdas de prueba para conocer la firma real que espera el grader. Dos funciones de la plantilla tenían una firma distinta a la que usan las pruebas: `get_loss(class_weights)` y `model_predict(model, X_test)`.
3. Reproducir el notebook fuera de Colab para confirmar los valores numéricos (pérdidas, pesos, predicciones).
4. Ejecutar las celdas de evaluación en Colab y corregir según la retroalimentación del grader de UNCode.

4. Qué se hizo, función por función

4.1 class_weights: peso de cada clase

Se cuentan las muestras de cada clase (n_0, n_1) y se aplica la fórmula del enunciado: $w_0 = (n_0 + n_1) / (2 \cdot n_0)$ y $w_1 = (n_0 + n_1) / (2 \cdot n_1)$. Con $n_0 = 50$ y $n_1 = 33$ se obtiene **$w_0 = 0.83$** y **$w_1 = 1.25758$**: un error en la clase minoritaria pesa 1.5 veces más.

```
def class_weights(y):
    import numpy as np
    n_0 = np.sum(y == 0)
    n_1 = np.sum(y == 1)
    w_0 = float((n_0 + n_1) / (2 * n_0))
    w_1 = float((n_0 + n_1) / (2 * n_1))
    weights_list = [w_0, w_1]
    return weights_list
```

4.2 get_loss: entropía cruzada ponderada

La pérdida es $L = -(1/N)\ \Sigma\ [\ w_1 \cdot y_i \cdot \log(\hat{y}_i) + w_0 \cdot (1 - y_i) \cdot \log(1 - \hat{y}_i)\ ]$. `get_loss` recibe los pesos y devuelve otra función `loss(y_true, y_pred)`, que es la firma que Keras exige (una *closure*). Las predicciones se recortan a $[\varepsilon, 1 - \varepsilon]$ para evitar log(0).

```
def get_loss(class_weights):
    import keras
    w_0, w_1 = class_weights
    def loss(y_true, y_pred):
        y_true = keras.ops.cast(y_true, y_pred.dtype)
        eps = keras.backend.epsilon()
        y_pred = keras.ops.clip(y_pred, eps, 1 - eps)
        w_bce = -keras.ops.mean(w_1 * y_true * keras.ops.log(y_pred)
                                + w_0 * (1 - y_true) * keras.ops.log(1 - y_pred))
        return w_bce
    return loss
```

Comprobación: con $\hat{y} = 0.5$ para todas las muestras la pérdida da $\log 2 = 0.6931$, porque el promedio ponderado de los pesos es exactamente 1: $(50 \cdot 0.83 + 33 \cdot 1.25758) / 83 = 1$.

4.3 optimizer: selección del optimizador

Devuelve el optimizador de `keras.optimizers` pedido (SGD, Adam o RMSprop) con la tasa de aprendizaje indicada.

```
def optimizer(type_opt, learning_rate):
    import keras
    if type_opt == 'SGD':
        opt = keras.optimizers.SGD(learning_rate=learning_rate)
    elif type_opt == 'Adam':
        opt = keras.optimizers.Adam(learning_rate=learning_rate)
    elif type_opt == 'RMSprop':
        opt = keras.optimizers.RMSprop(learning_rate=learning_rate)
    else:
        raise ValueError("type_opt debe ser 'SGD', 'Adam' o 'RMSprop'")
    return opt
```

4.4 get_model: la regresión logística

Una regresión logística equivale a una red de una sola capa Dense con activación sigmoide: 13 pesos más 1 sesgo = **14 parámetros**, igual que el resumen esperado. El modelo se compila con el optimizador y la pérdida ponderada.

```
def get_model(n_inputs, n_neurons, optimizer, loss):
    import keras
    model = keras.models.Sequential([
        keras.layers.Input(shape=(n_inputs,)),
        keras.layers.Dense(n_neurons, activation='sigmoid')
    ])
    model.compile(optimizer=optimizer, loss=loss)
    return model
```

4.5 train_model: entrenamiento

Esta fue la decisión clave. Se probaron dos formas de entrenar partiendo de pesos en cero:

Método	Pérdida época 0	¿Coincide con lo esperado (0.9399663)?
Bucle manual con GradientTape (1 paso por época, todo el lote)	0.6931	No
model.fit (lotes de 32, 3 pasos por época)	0.9399663	Sí

Se usó `model.fit`. El optimizador y la pérdida no se pasan de nuevo porque ya quedaron dentro del modelo al compilarlo.

```
def train_model(model, epochs, optimizer, X_t, Y_t, loss):
    history = model.fit(X_t, Y_t, epochs=epochs, verbose=0)
    losses = history.history['loss']
    return losses, model.get_weights()
```

4.6 model_predict: predicciones

Al llamar al modelo directamente se obtiene un tensor de tamaño (l, 1) con la probabilidad de que cada vino sea de la clase 1. Redondeando con umbral 0.5 se obtiene la clase predicha.

```
def model_predict(model, X_test):
    import tensorflow as tf
    y_pred = model(tf.convert_to_tensor(X_test, dtype=tf.float32))
    return y_pred
```

5. Ajustes exigidos por el grader de UNCode

Al ejecutar las pruebas en Colab aparecieron dos problemas que no se ven en la ejecución local:

Síntoma en el grader	Causa	Corrección
Error: *name 'tf' is not defined* (nota 0)	El grader ejecuta las funciones en su propio espacio y no ve los import del notebook.	Importar numpy, tensorflow y keras dentro de cada función calificada.
El tipo de dato retornado al llamar loss no es el esperado (nota 10)	La primera versión forzaba todo a float32 con tf.cast, y el grader espera el tipo de dato de y_pred.	Reescribir la pérdida con keras.ops (como sugiere la Pista 1). Pasó con nota 100.

6. Resultados

6.1 Pruebas del grader

Prueba	Función evaluada	Nota
Test 1_1 y 1_2	class_weights	100 / 100
Test 2_1 y 2_2	get_loss	100 / 100
Test 3_1 y 3_2	optimizer	100 / 100
Test 4_1 y 4_2	get_model	100 / 100
Test 5_1 y 5_2	train_model	100 / 100
Test 6_1 y 6_2	model_predict	100 / 100

6.2 Salidas obtenidas frente a las esperadas

Verificación	Esperado	Obtenido
Pesos de clase	[0.83, 1.25758]	[0.83, 1.25758]
Pérdida con ŷ = 0.5	0.6931471	0.6931471
Parámetros del modelo	14	14
Pérdida época 0 / época 1	0.9399663 / 0.3858068	0.9399663 / 0.3858068
w[0] y b tras 10 épocas	2.2023726 / −0.0991320	2.2023726 / −0.0991321
Primeras 2 predicciones	0.9925333 / 0.0018804	0.9925333 / 0.0018804
Accuracy sobre X_t	0.9879518	0.9879518

Las diferencias en el séptimo decimal de b se deben a la versión de TensorFlow (Colab usa 2.21; el curso se hizo con 2.18).

6.3 Evolución de la pérdida

La pérdida baja de 0.94 a 0.04 en 10 épocas. La mayor caída ocurre en las primeras 4 épocas; después el modelo ya separa casi todas las muestras y la mejora es marginal.

Época	1	2	3	4	5	6	7	8	9	10
Pérdida	0.940	0.386	0.243	0.106	0.105	0.068	0.045	0.042	0.042	0.038

6.4 Matrices de confusión

Filas = clase real, columnas = clase predicha.

	Entrenamiento: pred. 0	Entrenamiento: pred. 1	Prueba: pred. 0	Prueba: pred. 1
Real 0 (vino tipo 2)	49	1	18	3
Real 1 (vino tipo 3)	0	33	0	15

7. Cómo leer los resultados

- **El modelo separa bien las dos clases.** Con solo 10 épocas, un modelo lineal acierta 98.8 % en entrenamiento y 91.7 % en prueba.
- **La ponderación funcionó.** Ningún vino de la clase minoritaria (tipo 3) fue mal clasificado, ni en entrenamiento ni en prueba: su recall es 100 %.
- **Los errores van en una sola dirección.** Los 3 errores de prueba son vinos tipo 2 clasificados como tipo 3. Es el costo esperado de darle más peso a la clase minoritaria: el modelo prefiere equivocarse hacia ella.
- **Hay una leve brecha entre entrenamiento y prueba** (≈7 puntos). Con 36 muestras de prueba, cada error equivale a 2.8 puntos, así que la diferencia son 3 vinos.
- **Nota sobre la celda 6:** aunque dice "Accuracy sobre X_test", en realidad evalúa sobre X_t (entrenamiento). El valor sobre el conjunto de prueba real es 91.7 %.

Variables con más peso en el modelo

Como los datos están escalados a [0, 1], la magnitud de cada peso indica cuánto influye la variable. Un peso positivo empuja hacia la clase 1 (vino tipo 3) y uno negativo hacia la clase 0 (vino tipo 2).

Variable	Peso	Favorece a
Flavanoids	-6.01	Clase 0 (tipo 2)
OD280/OD315	-5.34	Clase 0 (tipo 2)
Color intensity	+5.08	Clase 1 (tipo 3)
Hue	-4.46	Clase 0 (tipo 2)
Malic acid	+3.88	Clase 1 (tipo 3)
Proanthocyanins	-3.08	Clase 0 (tipo 2)

Los vinos tipo 3 se caracterizan por mayor intensidad de color y ácido málico, y menos flavonoides, OD280/OD315 y tono (hue).

8. Pasos para reproducir y entregar

1. Guardar una copia del notebook en Drive (Colab indica "No se pueden guardar cambios" en el original).
2. Ejecutar las celdas de conexión e iniciar sesión en UNCode.
3. Ejecutar las celdas de carga de datos y luego las seis funciones con sus pruebas, en orden.
4. Verificar que cada `grader.run_test(...)` muestre "Your test grade: 100".
5. Ejecutar `grader.submit_task(globals())` para la entrega oficial y revisarla en UNCode.